Loop Analysis Examples

8th August 2024

1 Introduction

This worksheet shows some examples of simple analysis of loops given an abstract model of a CPU. For all the examples shown here, we assume we are running on a super-scalar CPU capabable of initiating at each cycle:

- a Load,
- a Store,
- a Floating Point Add,
- a Floating Point Multiply.
- 2 Integer operations or branch

Additionally, we assume that

- Memory accesses are to L1 cache
- Looping is essentially free
- Incremenent loop counters, and array addresses such as a[i + 1] are free
- Local variables used within loops are retained in registers (don't need to be loaded/stored)

For the purposes of these examples, we use for latencies (approximately the same as Gadi)

- 3 cycles for Load/Store
- 4 cycles for Floating Point Add/Multiply

2 Dot Product Example

We start with a simple dot product example which was covered in lectures.

```
float A[N], B[N];
int i;
float s = 0.0;

for (i = 0; i < N; i ++) {
  s = s + A[i] * B[i];
}
```

2.1 Without Pipelining

From the loop, we can see that each iteration we need to

1. Load A[i]
2. Load B[i]
3. Multiply A[i] * B[i]
4. Add s + A[i] * B[i]

As `s` is a local variable, we don't need to load and store it as we assume it will be stored in a register during the loop. To understand the number of cycles we arrange these instructions into a table with one line per cycle, making note of when instructions finish so that subsequent instructions with dependencies are only started once results/resources they need are available.

Cycle	Load	Add	Mult	Store	Completed
1	Load A[i]				
2					
3					
4	Load B[i]				Load A[i], Add s + A[i-1] * B[i-1]
5					
6					
7			Mult. A[i] * B[i]		Load B[i]
8					
9					
10					
11		Add s + A[i] * B[i]			Mult. A[i] * B[i]

From this analysis there are 11 cycles per iteration. Some things to note in this example

- In the absence of pipelined units, we have to do one load at a time so for example, Load B[i] has to wait until Load A[i] is finished. This is a resource dependency due to there only being one Load unit which doesn't support pipelining.
- In cycle 11, we do not need to wait for the Add to complete before looping. A small caveat here is that we need to ensure unfinished instructions upon looping are not dependencies for early instructions. For this reason it is good practice to show in the completed column instructions from the previous loop, e.g. the add of the previous iteration shown in red finishes at cycle 4 and in this case the result, `s`, is not needed again until cycle 11 so the dependencies are satisfied.

2.2 With two times loop unrolling

We can analyze the same loop with loop unrolling. The extra iterations (for example if N is odd) are not important for loop analysis and are ignored.

```
float A[N], B[N];
int i;
float s = 0.0;

for (i = 0; i < N; i += 2) {
  s = s + A[i] * B[i] + A[i + 1] * B[i + 1];
}
```

Cycle	Load	Add	Mult	Store	Completed
1	Load A[i]				
2					
3					
4	Load B[i]				Load A[i], Add s + A[i-1] * B[i-1]
5					
6					
7	Load A[i + 1]		Mult. A[i] * B[i]		Load B[i]
8					
9					
10	Load B[i + 1]				Load A[i + 1]
11		Add s + A[i] * B[i]			Mult. A[i] * B[i]
12					
13			Mult. A[i + 1]*B[i + 1]		Load B[i + 1]
14					
15					Add s + A[i] * B[i]
16					
17		Add s + A[i + 1]*B[i + 1]			Mult. A[i + 1]*B[i + 1]

This shows we have 17 cycles for 2 iterations or 8.5 cycles per iteration. The main benefit of loop unrolling is in being able to take advantage of the superscalar architecture to execute some operations in parallel/overlapping. For example at cycle 7, we are able to start the first multiplication while also starting the load of the next `A` value.

2.3 With Pipelining

Assume now we have pipeline depths for our architecture that matches the latencies given above. This means that our operations (Load,Store,Add,Mult.) take the same amount of time, but that we can start independent operations of the same type at the next cycle. First consider the original version.

Cycle	Load	Add	Mult	Store	Completed
1	Load A[i]				
2	Load B[i]				
3					
4					Load A[i], Add s + A[i-1] * B[i-1]
5			Mult. A[i] * B[i]		Load B[i]
6					
7					
8					
9		Add s + A[i] * B[i]			Mult. A[i] * B[i]

The consequence of pipelining means that the two loads can occur one after the other, and that the two operands to the multiply operation are available 2 cycles earlier so we now have 9 cycles per iteration instead of 11.

2.4 With Pipelining and Loop Unrolling

Cycle	Load	Add	Mult	Store	Completed
1	Load A[i]				
2	Load B[i]				
3	Load A[i + 1]				
4	Load B[i + 1]				Load A[i], Add s + A[i] * B[i]
5			Mult. A[i] * B[i]		Load B[i]
6					Load A[i + 1]
7			Mult. A[i + 1]*B[i + 1]		Load B[i + 1]
8					
9		Add s + A[i] * B[i]			Mult. A[i] * B[i]
10					
11					Mult. A[i + 1] * B[i + 1]
12					
13		Add s + A[i + 1]*B[i + 1]			Add s + A[i] * B[i]

With pipelining and two times loop unrolling we obtain 13 cycles per two iterations or 6.5 cycles per iteration. The main factor inhibiting optimal performance is the dependency on the local variable `s`. The second multiply finishes at cycle 11, but its result can't be added to `s` until cycle 13. Two improve performance, we could use two varaibles `s1` and `s2` as shown in lectures to break this dependency and reduce the cycles to 11.

3 SAXPY

The common BLAS function saxpy can be implemented (slightly simplified) as follows

```
void saxpy(int N, float a, float *x, float *y)
{
  int i;
  for (i = 0; i < N; i ++) {
    y[i] = y[i] + a*x[i];
  }
}
```

3.1 With Pipelining

Cycle	Load	Add	Mult	Store	Completed
1	Load x[i]				
2	Load y[i]				
3					Store y[i - 1]
4			Mult. a*x[i]		Load x[i]
5					Load y[i]
6					
7					
8		Add y[i] + a*x[i]			Mult. a*x[i]
9					
10					
11					
12				Store y[i]	Add y[i] + a*x[i]

This gives 12 cycles per iteration.

3.2 With pipelining and two times unrolling

```
void saxpy(int N, float a, float *x, float *y)
{
  int i;
  for (i = 0; i < N; i += 2) {
    y[i] = y[i] + a*x[i];
    y[i + 1] = y[i] + a*x[i + 1];
  }
}
```

Cycle	Load	Add	Mult	Store	Completed
1	Load x[i]				Store y[i - 2]
2	Load y[i]				
3	Load x[i + 1]				Store y[i - 1]
4	Load y[i + 1]		Mult. a*x[i]		Load x[i]
5					Load y[i]
6			Mult. a*x[i + 1]		Load x[i + 1]
7					
8		Add y[i] + a*x[i]			Mult. a*x[i]
9					
10		Add y[i + 1] + a*x[i + 1]			Mult. a*x[i + 1]
11					
12				Store y[i]	Add y[i] + a*x[i]
13					
14				Store y[i + 1]	Add y[i + 1] + a*x[i + 1]

We now have 14 cycles for 2 iterations or 7 cycles per iteration which would give nearly twice the performance.